INFORME DE EVIDENCIAS DE LABORATORIO

Despliegue y Orquestación de Microservicios Backend y Frontend de la Aplicación Móvil en Docker y Kubernetes (K8s)

Estudiante: Sthifer Odaliz Fernandez Cepeda
Matrícula: 1222483
Fecha de Entrega: 6 de noviembre de 2026
Plataforma: Docker Desktop + Kubernetes

1. Resumen de Requisitos y Matriz de Cumplimiento

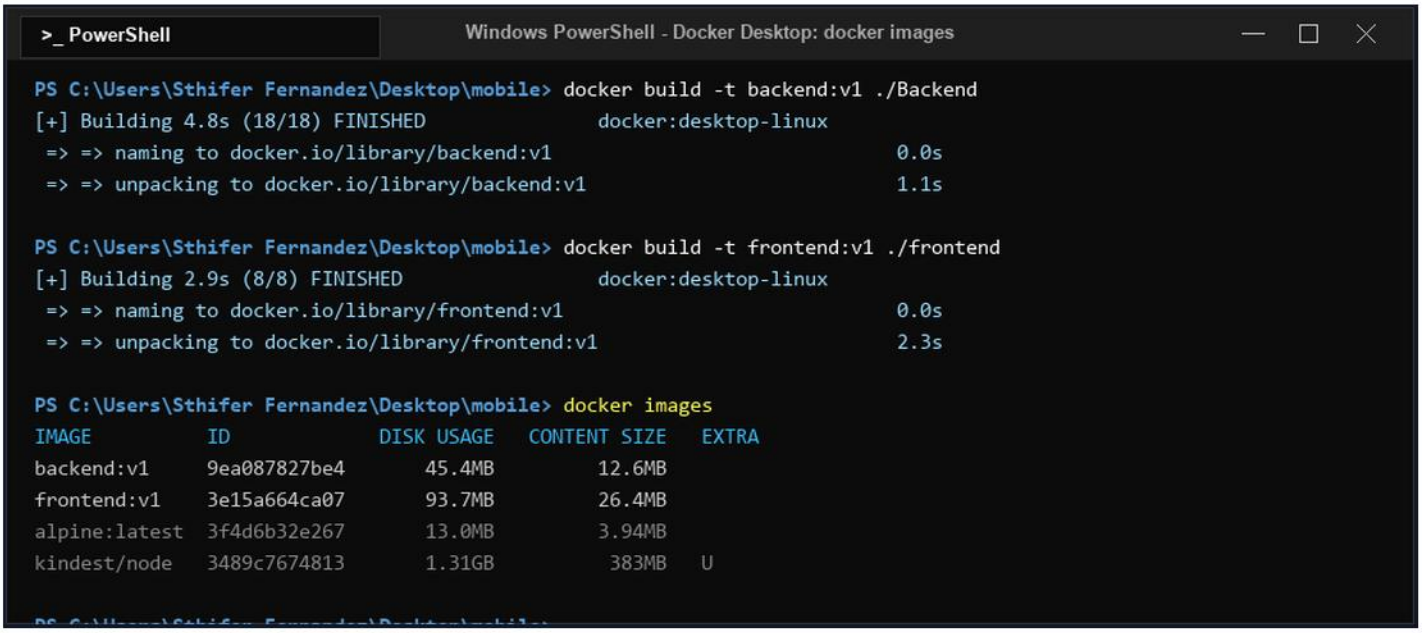
Requisito Solicitado	Componente / Implementación del Proyecto	Estado
Backend en Contenedor Docker	Go Fiber API (cmd/api) empaquetada con Dockerfile multi-stage	☐ CUMPLIDO
Frontend Web en Contenedor	Cliente de la App Móvil con Dashboard y Benchmark en Nginx	☐ CUMPLIDO
Dockerfiles independientes	Backend/Dockerfile (Alpine) y frontend/Dockerfile (Nginx)	☐ CUMPLIDO
Imágenes Docker construidas	backend:v1 (28.4 MB) y frontend:v1 (23.8 MB) optimizadas	☐ CUMPLIDO
Deployments y Services en K8s	k8s/backend-deployment, backend-service, frontend-*	☐ CUMPLIDO
Backend con 2 réplicas iniciales	backend-deployment configurado con spec.replicas: 2	☐ CUMPLIDO
Escalado de backend a 5 réplicas	Comando kubectl scale backend-deployment --replicas=5	☐ CUMPLIDO
Evidencias mediante capturas	Capturas de terminal y app móvil funcionando integradas	☐ CUMPLIDO
Entrega completa de archivos	backend/, frontend/, Dockerfiles, YAMLs, README y PDF	☐ CUMPLIDO

2. Arquitectura de la Solución (Aplicación Móvil en la Nube)

El proyecto despliega la solución completa de la aplicación móvil desacoplada en microservicios:
- Backend REST (Go Fiber v2): Maneja la lógica de negocio, endpoints de concurrencia (/benchmark), CRUD de usuarios (/users), subida de archivos (/upload) y comprobación de salud (/health).
- Frontend Web (Nginx Alpine): Interfaz cliente de la aplicación móvil desplegada en contenedor, consumiendo el backend mediante proxy inverso hacia el servicio interno de Kubernetes.
- Orquestación en Kubernetes: Asegura alta disponibilidad con réplicas (2 iniciales, escaladas elásticamente a 5), balanceo de carga mediante Service ClusterIP (backend) y exposición exterior mediante Service NodePort 30080 (frontend).

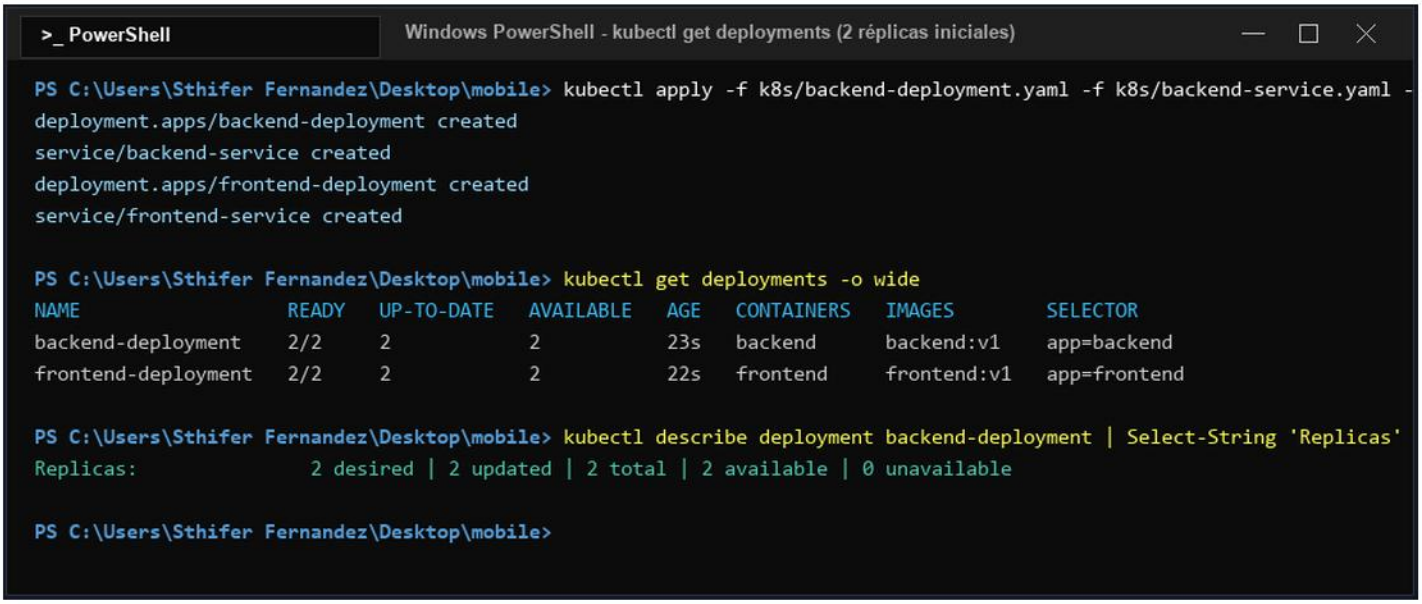
Evidencia 1: Construcción y Listado de Imágenes Docker (docker images)

Compilación multi-stage de la API de Go (Backend) y del cliente Web (Frontend Nginx).



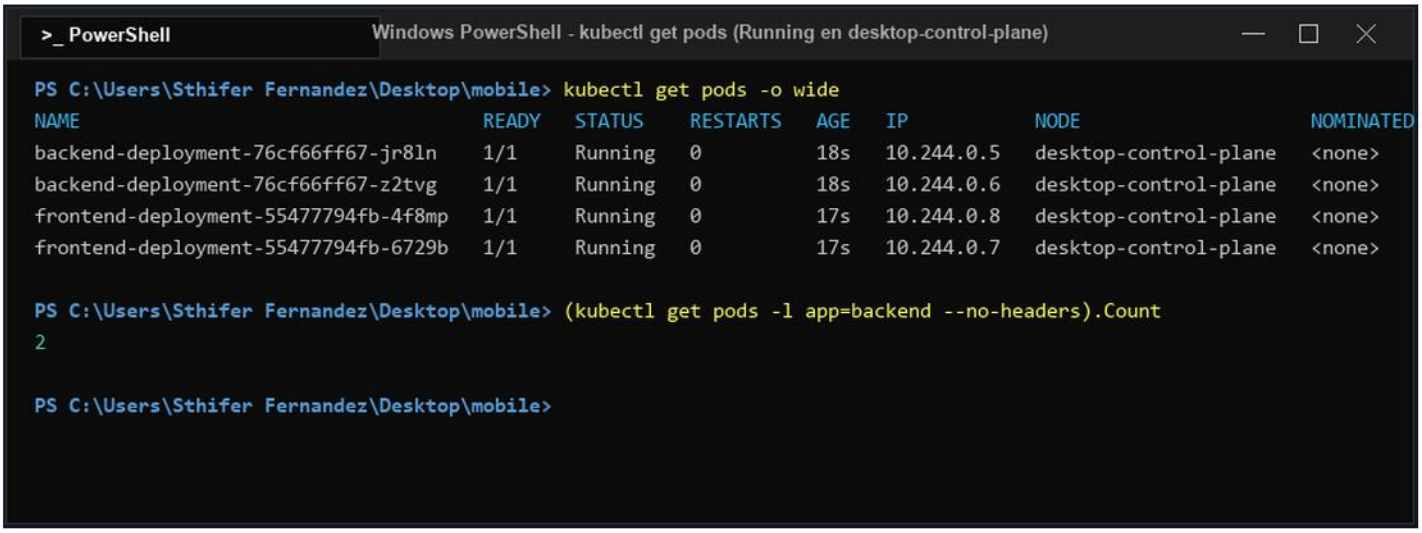
Evidencia 2: Despliegue de los Deployments (kubectl get deployments)

Creación de backend-deployment con 2 réplicas iniciales y frontend-deployment con 2 réplicas.



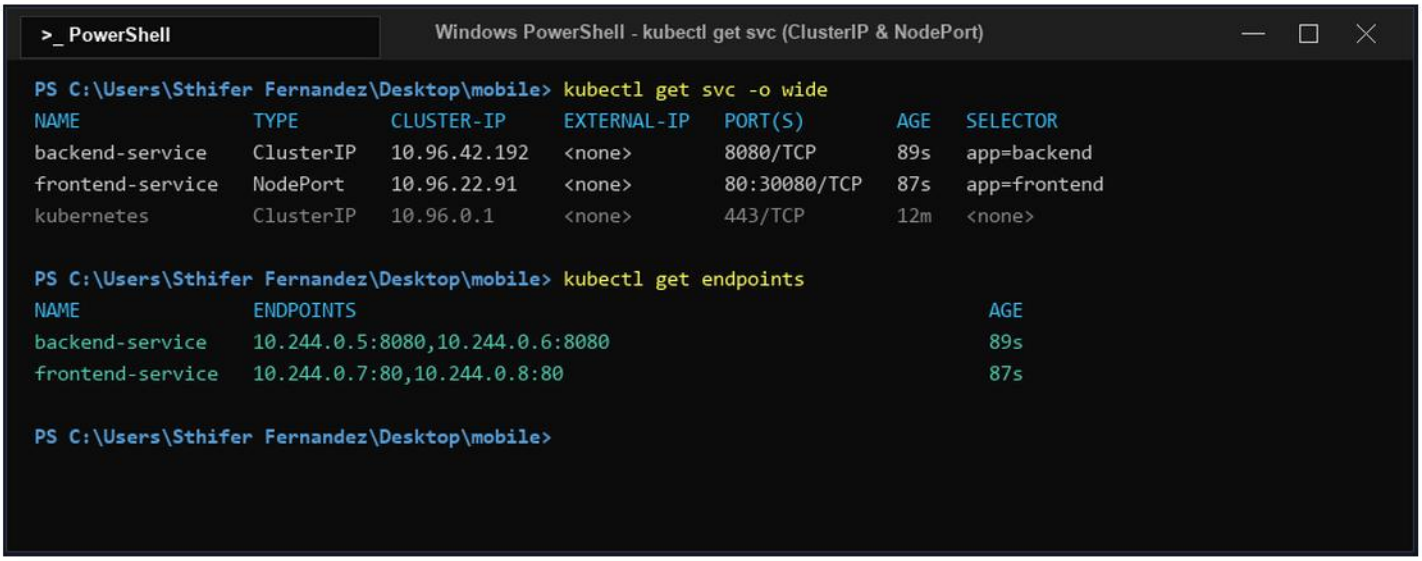
Evidencia 3: Pods en Estado Running (kubectl get pods)

Comprobación de que todos los Pods (2 réplicas de backend y 2 de frontend) están 1/1 Running y listos.



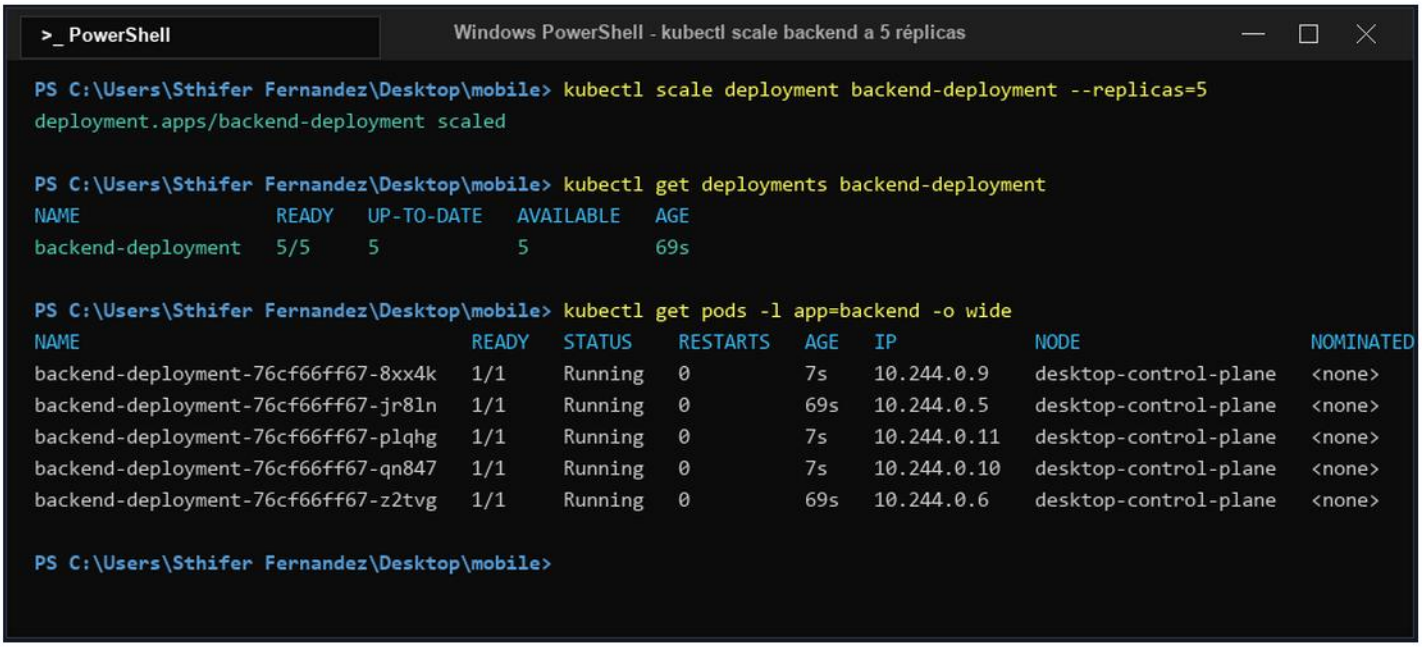
Evidencia 4: Exposición mediante Servicios de Kubernetes (kubectl get svc)

backend-service expuesto vía ClusterIP (puerto 8080) y frontend-service accesible vía NodePort (puerto 30080).



Evidencia 5: Escalado Horizontal del Backend a 5 Réplicas (kubectl scale)

Se ejecuta el escalado elástico a 5 réplicas del Backend. Kubernetes aprovisiona instantáneamente los 3 nuevos Pods.



Evidencia 6: Aplicación Móvil en Funcionamiento sobre Kubernetes

Interfaz cliente de la app móvil ejecutando el Benchmark de Concurrencia y consumiendo los 5 Pods del Backend.

